MANUAL TÉCNICO DEL SISTEMA POS

SENA PDV

1. RESUMEN GENERAL DEL SISTEMA

El sistema **SENA PDV (Punto de Venta e Inventarios)** es una solución tecnológica integral diseñada para el control y gestión en tiempo real de operaciones comerciales de venta, compra y administración de inventarios.

Nombre del Sistema	SENA PDV (Punto de Venta e Inventarios)
Lenguaje Principal	Python 3.x
Framework GUI	Flet (basado en Flutter para Python)
Base de Datos	MySQL (Base de datos relacional sena_pdv)
Patrón Arquitectónico	Modelo - Vista - Controlador (MVC)
Formatos de Entrega	Código fuente, Ejecutable (.exe), Base de datos SQL, Manuales

Propósito y Alcance del Sistema:

- **Gestión POS y Facturación:** Emisión de ventas en tiempo real con generación de facturas, tirillas térmicas e impresiones en formato PDF.
- **Registro de Compras:** Ingreso documentado de mercancía desde proveedores con actualización directa al stock.
- **Emisión de Remisiones:** Control de despachos y salidas de inventario sin afectación directa de transacciones monetarias.
- **Control de Inventario Automático:** Actualización instantánea de existencias y emisión de alertas por bajo stock.
- **Consultas y Reportes Consolidados:** Módulo de búsqueda unificada de documentos e historial comercial con exportación de reportes a Excel.
- **Gestión de Usuarios y Seguridad:** Control de acceso centralizado basado en roles de usuario (Administrador, Cajero, etc.).

2. ARQUITECTURA DE SOFTWARE Y ESTRUCTURA DE DIRECTORIOS

El sistema implementa una arquitectura **Modelo-Vista-Controlador (MVC)** modularizada y desacoplada, garantizando mantenibilidad y escalabilidad. A continuación, se detalla la estructura física del proyecto entregado:

```
Proyecto/
└── App/
    ├── main.py                     # Punto de entrada principal de la aplicacion
    ├── requirements.txt            # Dependencias del proyecto
    │
    ├── database/                   # Capa de Conexion a Base de Datos
    │   ├── __init__.py
    │   ├── bd.py                   # Conector MySQL (mysql.connector)
    │   └── database.sql            # Sql de la base de datos
    │
    ├── models/                     # Capa de Datos (Modelos - Consultas SQL)
    │   ├── __init__.py
    │   ├── loginModel.py           # Autenticacion de usuarios
    │   ├── usuariosModel.py        # Creacion y gestion de usuarios
    │   ├── ventasModel.py          # CRUD de ventas, clientes, productos y detalle
    │   ├── comprasModel.py         # CRUD de compras, proveedores y detalle
    │   ├── remisionesModel.py      # CRUD de remisiones y detalle
    │   ├── inventarioModel.py      # Consultas de inventario y alertas de stock
    │   └── facturasModel.py        # Consultas generales de documentos e informes
    │
    ├── controllers/                # Capa de Logica de Negocio (Controladores)
    │   ├── __init__.py
    │   ├── ventasController.py     # Calculos de ventas, formateos y confirmacion
    │   ├── comprasController.py    # Reglas para registro de compras
    │   ├── remisionController.py   # Logica para gestion de remisiones
    │   ├── facturasController.py   # Preparacion de datos para vistas/reportes
    │   └── usuarioController.py    # Logica de validacion de usuarios
    │
    ├── views/                      # Capa de Interfaz de Usuario (Vistas Flet)
    │   ├── __init__.py
    │   ├── inicio.py               # Pantalla de bienvenida
    │   ├── login.py                # Formulario de autenticacion
    │   ├── menu.py                 # Panel principal (Dashboard / Menu)
    │   ├── vender.py               # Ventana de Punto de Venta (POS)
    │   ├── comprar.py              # Ventana de Registro de Compras
    │   ├── remision.py             # Ventana de Remisiones
    │   ├── inventario.py           # Ventana de Inventario (Bajo stock)
    │   ├── consultarFacturas.py    # Historial de documentos y exportacion
    │   └── crearUsuario.py        # Registro de nuevos usuarios
    │
    ├── routers/                    # Capa de Enrutamiento
    │   ├── __init__.py
    │   └── navegacion.py           # Gestor central de rutas de Flet
    │
    └── utils/                      # Componentes Reutilizables y Utilidades
        ├── __init__.py
        ├── abrirNuevaVentana.py    # Lanzador de vistas en procesos independientes
        ├── buscarCliente.py        # Modal/dialogo para busqueda de clientes
        ├── buscarProducto.py       # Modal/dialogo para busqueda de productos
        ├── buscarProveedor.py      # Modal/dialogo para busqueda de proveedores
        ├── crearP_C.py             # Dialogos de creacion rapida (Producto/Cliente)
        ├── confirmacion.py         # Dialogos modales de alerta/confirmacion
        ├── metodoPago.py           # Dialogo para desglose de pagos y cambio
        ├── tablaDocumentos.py      # Componentes de tablas interactivas
        ├── generarFactura.py       # Generador PDF (ReportLab) e impresion termica
        └── InformeExcel.py         # Exportacion de reportes a Excel (OpenPyXL)
```

3. MODELO DE BASE DE DATOS (ESQUEMA ENTIDAD-RELACIÓN)

La base de datos relacional **sena_pdv** en MySQL consta de 10 tablas interrelacionadas mediante llaves foráneas para garantizar la integridad referencial de los datos de negocio:

Tabla	Campos Principales	Descripción / Claves
usuarios	id_usuario (PK), nombre, usuario (UNIQUE), password, rol	Gestión de credenciales y roles de acceso
clientes	id_cliente (PK), nombre, documento, direccion, telefono, rol	Directorio de clientes para ventas/remisiones
proveedores	id_proveedor (PK), nombre	Catálogo de proveedores de compras
producto	id_producto (PK), nombre, unidad, lugar, precio, stock	Catálogo maestro de inventario y precios
venta	id_venta (PK), fecha, total, metodo_pago, estado, id_cliente (FK), id_usuario (FK), dinero_recibido, cambio, notas	Encabezados de transacciones de venta POS
detalle_venta	id_detalle (PK), id_venta (FK), id_producto (FK), cantidad, precio_unitario, subtotal	Detalle línea a línea de productos vendidos
compras	id_compra (PK), fecha, total, id_proveedor (FK), id_usuario (FK), estado	Encabezados de orden e ingreso de compras
detalle_compras	id_detalle (PK), id_compra (FK), id_producto (FK), cantidad, precio_unitario, subtotal	Detalle línea a línea de mercancía comprada
remision	id_remision (PK), fecha, tipo_remision, id_cliente (FK), id_usuario (FK), notas, estado	Encabezados de salidas/despachos de stock
detalle_remision	id_detalle_remision (PK), id_remision (FK), id_producto (FK), cantidad	Detalle de artículos en remisiones de salida

4. FLUJO DE DATOS Y LÓGICA DE NEGOCIO PRINCIPAL

A. Módulo de Ventas (POS)

- **1. Selección de Entidades:** Búsqueda interactiva de clientes y productos mediante ventanas modales (buscarCliente.py, buscarProducto.py).
- **2. Cálculo Dinámico:** El controlador ventasController.py recalcula dinámicamente subtotales, impuestos y monto total.
- **3. Liquidación de Pago:** El modal metodoPago.py registra formas de pago (Efectivo, Transferencia, Mixto) y liquida automáticamente el cambio sobrante.
- **4. Transacción ACID en Base de Datos (ventasModel.guardar_factura):** Inserta registro en 'venta', genera detalle en 'detalle_venta', y ejecuta la actualización de stock: UPDATE producto SET stock = stock - %s WHERE id_producto = %s. Aplica commit() o rollback() ante cualquier fallo.
- **5. Salida de Documentos:** Generación de factura digital en formato PDF mediante ReportLab (generarFactura.py) y comando de impresión a tirilla térmica POS.

B. Módulo de Compras e Ingreso de Stock

- **1. Captura de Proveedor:** Selección de proveedor e ingreso de renglones de compra con costo unitario.
- **2. Actualización de Inventario (comprasModel.guardar_compra):** Registra la transacción en 'compras' y 'detalle_compras', e incrementa de forma automática el inventario físico: UPDATE producto SET stock = stock + %s WHERE id_producto = %s.

C. Módulo de Remisiones

- **1. Despacho sin Cobro:** Permite registrar traslados o salidas de mercancía autorizadas asignadas a un cliente.
- **2. Descuento en Stock:** Se registra en 'remision' y 'detalle_remision', descontando unidades del stock sin alterar saldos contables de venta directa.

D. Arquitectura Multiventana y Subprocesos Independientes

- **1. Aislamiento de Procesos:** Mediante abrirNuevaVentana.py, el sistema puede lanzar vistas independientes invocando una nueva instancia de Python (sys.executable App/main.py).
- **2. Mantenimiento de Sesión:** Inyecta variables de entorno (POS_VIEW, POS_USUARIO) para preservar el contexto de autenticación y la vista requerida sin interferir con la ventana principal.

5. DEPENDENCIAS Y REQUISITOS TÉCNICOS

Nota importante: Las siguientes dependencias de software solo son requeridas si el sistema se ejecuta desde el código fuente. La versión ejecutable (**.exe**) empaqueta e incluye la totalidad de estas librerías de forma nativa.

Librería / Módulo	Versión	Función en el Sistema
flet	0.85.2	Framework principal GUI para la interfaz gráfica reactiva
mysql-connector-repackaged	0.3.1	Driver oficial para conexión y consultas a MySQL
reportlab	4.5.1	Motor para maquetación e impresión de facturas en PDF
python-escpos	3.0	Comunicación e impresión directa en impresoras térmicas POS
openpyxl	3.1.5	Generación y exportación de reportes tabulares en Excel

6. GUÍA DE INSTALACIÓN Y PUESTA EN MARCHA

6.1 Requisitos Previos de Infraestructura

- **Sistema Operativo:** Windows 10 u 11 (Arquitectura de 64 bits).
- **Servidor de Base de Datos:** MySQL Server v5.7 o v8.0 en ejecución sobre el puerto 3306 (instalable vía Laragon).
- **Almacenamiento:** Espacio libre mínimo de 500 MB para ejecutable, base de datos y repositorio local de PDF/Excel.
- **Hardware de Impresión:** Impresora térmica de tickets POS instalada como predeterminada o mediante puerto USB/COM.

6.2 Contenido de la Entrega

El paquete comprimido entregado al cliente contiene los siguientes elementos estructurados:

- **Código Fuente Completo:** Directorio App/ estructurado con módulos Python.
- **Aplicación Ejecutable (.exe):** SENA_PDV.exe listo para producción.
- **Base de Datos SQL:** Archivo database.sql con la estructura y datos de inicio.
- **Documentación:** Manual Técnico y Manual de Usuario final en formato digital.

6.3 Instalación y Configuración de la Base de Datos

- **1. Verificación del Servicio:** Iniciar el servidor MySQL mediante Laragon y comprobar disponibilidad en el puerto 3306.
- **2. Importación del Script SQL:** Importar el archivo database.sql a través de phpMyAdmin, MySQL Workbench o mediante el comando de consola: mysql -u root -p < database.sql.

- **3. Personalización del Nombre (Opcional):** Para renombrar la base de datos, modifique las primeras líneas del script SQL antes de importarlo: CREATE DATABASE IF NOT EXISTS nombre_personalizado; USE nombre_personalizado;

6.4 Configuración de la Conexión a la Base de Datos

Los parámetros de conexión se gestionan centralizadamente en el archivo **App/database/bd.py** mediante lectura de variables de entorno con fallback a valores por defecto:

```
DB_HOST = os.getenv("DB_HOST", "localhost")
DB_USER = os.getenv("DB_USER", "root")
DB_PASSWORD = os.getenv("DB_PASSWORD", "")
DB_NAME = os.getenv("DB_NAME", "sena_pdv")
```

6.5 Puesta en Marcha mediante la Aplicación Ejecutable (.exe)

- **1. Lanzamiento:** Ubicarse en la carpeta entregada y hacer doble clic sobre SENA_PDV.exe.
- **2. Autenticación:** Ingresar credenciales de usuario configuradas en la tabla 'usuarios' (ej. usuario por defecto).
- **3. Navegación:** El sistema desplegará el menú principal (Dashboard) permitiendo el acceso directo a los módulos operacionales.

6.6 Ejecución desde Código Fuente (Soporte Técnico / Desarrollo)

- **1. Requisito de Entorno:** Instalar Python 3.10 o superior (verificar en terminal mediante python --version).
- **2. Creación e Instalación de Virtualenv:**

```
# En Windows (PowerShell)
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r App/requirements.txt

# Ejecución del sistema
python App/main.py
```

6.7 Diagnóstico y Solución de Problemas Frecuentes

- **Error de Conexión a Base de Datos:** Asegúrese de que el servicio MySQL esté activo en Laragon/XAMPP y que el puerto 3306 esté abierto. Verifique usuario y contraseña en bd.py.
- **La Aplicación no Abre:** Revise que no existan otros procesos SENA_PDV.exe colgados en el Administrador de Tareas. Confirme soporte para arquitectura x64.
- **Fallo de Impresión Térmica:** Verifique la conexión USB/COM de la impresora y confirme que el controlador esté configurado como impresora predeterminada.

7. SOPORTE Y MANTENIMIENTO

Para asistencia técnica especializada, soporte correctivo o solicitudes de nuevas características, póngase en contacto con el equipo de desarrollo adjuntando el código de error, módulo afectado y captura de pantalla correspondiente.